

CodeGen

Group 8

**SQL to MongoDB Conversion using
Small Language Models with
Execution-Based Evaluation**

Project Completion Report

Team Members:

Chandrashekar

Jaya Krishna

Pavan Y

Swathi

Professor: Dr. Anil Neelkanti

Mentors: Nayan, Himanshu

**International Institute of Information Technology
Hyderabad**

January 24, 2026

Contents

1	Abstract	2
2	Introduction	3
2.1	Project Title	3
2.2	Models Used (Local Inference)	3
2.3	Key Definitions	3
3	Phase I: Text-to-SQL Generation	5
3.1	Phase I Algorithm (Text-to-SQL)	5
3.2	Motivation	7
3.3	Execution-Based Validation	7
3.3.1	Evaluation and Results (1000 samples)	7
3.4	Phase I: Paper-Style Algorithm (Text-to-SQL)	8
3.5	Sanitization and Robustness	8
3.6	Why Strict Exact Match is Hard for Mongo Pipelines	8
4	Execution-Based Verification	9
4.1	Verification Strategy	9
4.2	Relaxed Execution Agreement (REA)	9
4.2.1	Jaccard Similarity	10
4.2.2	Why Jaccard Similarity is Appropriate	10
5	Definitions and Terminology	11
6	Results and Analysis	12
6.1	Strict MongoDB Exact Match Accuracy	12
6.2	Subset Evaluation (100–200)	12
6.3	Scaled Evaluation (1–2500)	12
6.4	Interpreting the Metrics	13
6.5	MLX Framework for Local Fine-Tuning	13
7	LoRA Fine-Tuning Experiments	14
7.0.1	LoRA Configuration and Training Setup	14
7.1	LoRA Training Procedure	15

7.2	Setup	17
7.3	Why LoRA is Required	17
7.4	Learning Rate and Loss Trend (2 Epochs)	17
7.5	Why We Stop at 2 Epochs	18
8	Gradio User Interface	19
8.1	User Workflow	19
8.2	Key Features Implemented	19
8.3	Why the UI Matters	20
8.4	Integration with the Core Pipeline	20
8.5	UI Workflow	20
8.6	Why the UI Matters	20
9	Challenges and Learnings	21
9.1	Challenges Faced	21
9.2	Key Learnings	21
10	Conclusion and Future Work	22
10.1	Conclusion	22
10.2	Future Scope: Agentic AI Enhancements	22

Chapter 1

Abstract

This project explores the use of Small Language Models (SLMs) for converting database questions and queries across paradigms, with a strong emphasis on *execution-based* correctness. The system is evaluated using the BIRD (Bird-Bench) dataset and leverages locally hosted language models, schema-aware Retrieval-Augmented Generation (RAG), and a robust execution verification framework.

We implement a two-phase pipeline: (i) Text-to-SQL generation and (ii) SQL-to-MongoDB translation. Unlike traditional approaches that rely on string matching or AST comparison, we validate correctness by executing SQL queries on SQLite and generated MongoDB queries on MongoDB, followed by logical result comparison. Through iterative improvements in schema retrieval, sanitization, execution controls, and LoRA fine-tuning, we improve generation stability and move toward practical end-to-end correctness.

Chapter 2

Introduction

Relational databases using SQL and document-oriented NoSQL databases such as MongoDB represent fundamentally different data models and query paradigms. SQL relies on normalized schemas and explicit joins, while MongoDB operates on documents and pipeline-based aggregation. Translating queries between these paradigms is non-trivial and often requires schema understanding, join-key reasoning, and query restructuring.

With the emergence of Large and Small Language Models, there is increasing interest in automating such translations. However, many systems evaluate correctness using syntactic similarity, which is insufficient for cross-paradigm translation. This project addresses the limitation by focusing on execution-based evaluation, ensuring that translated queries are not only syntactically valid but semantically equivalent.

2.1 Project Title

Text-to-SQL→SQL-to-MongoDB Conversion using Small Language Models (SLMs) (BIRD / Bird-Bench dataset)

2.2 Models Used (Local Inference)

We use **Qwen2.5** (via **Ollama**) as the primary SLM for both stages:

- **Text-to-SQL:** Qwen2.5 via Ollama, executed locally on Mac M4 Pro.
- **SQL-to-MongoDB:** Qwen2.5 with schema-aware prompting and retrieved context.

2.3 Key Definitions

To keep the report self-contained, we define key terms used throughout:

- **SLM (Small Language Model):** A relatively compact instruction-tuned model that can run locally with acceptable latency.

Combined Architecture: Text-to-SQL - MongoDB Architecture

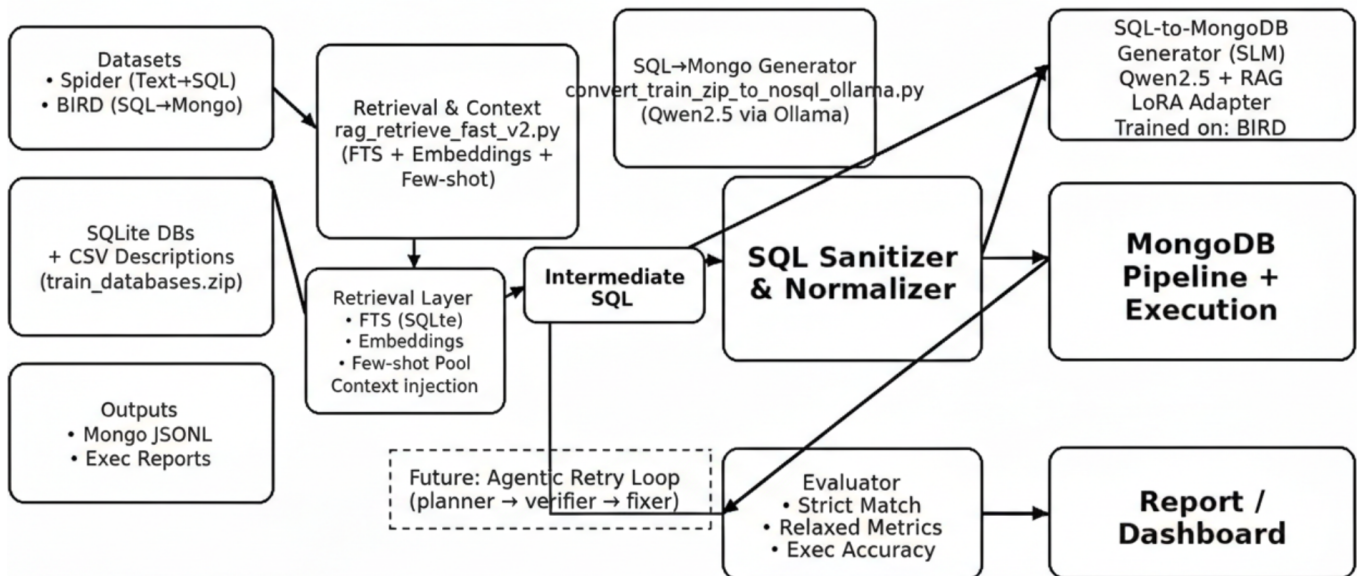


Fig. 1. Combined architecture: Text-to-SQL to SQL-to-MongoDB pipeline.

- **RAG (Retrieval-Augmented Generation):** A method that retrieves relevant context (schema, docs, examples) and injects it into the model prompt.
- **FTS5:** SQLite Full-Text Search index used for fast lexical retrieval when exact keyword or identifier matching is needed.
- **Executability vs Accuracy:** Executability measures whether a query runs without errors; accuracy measures whether results match the intended semantics.
- **Strict vs Relaxed Matching:** Strict matching requires an exact pipeline match; relaxed matching evaluates equivalence by comparing execution outputs.
- **LoRA:** A parameter-efficient fine-tuning method that learns low-rank adapters, improving task-following without retraining all model weights.

Chapter 3

Phase I: Text-to-SQL Generation

3.1 Phase I Algorithm (Text-to-SQL)

Phase I follows an execution-aware Text-to-SQL procedure where the generated SQL is treated as an executable program and validated on SQLite before it is forwarded to Phase II.

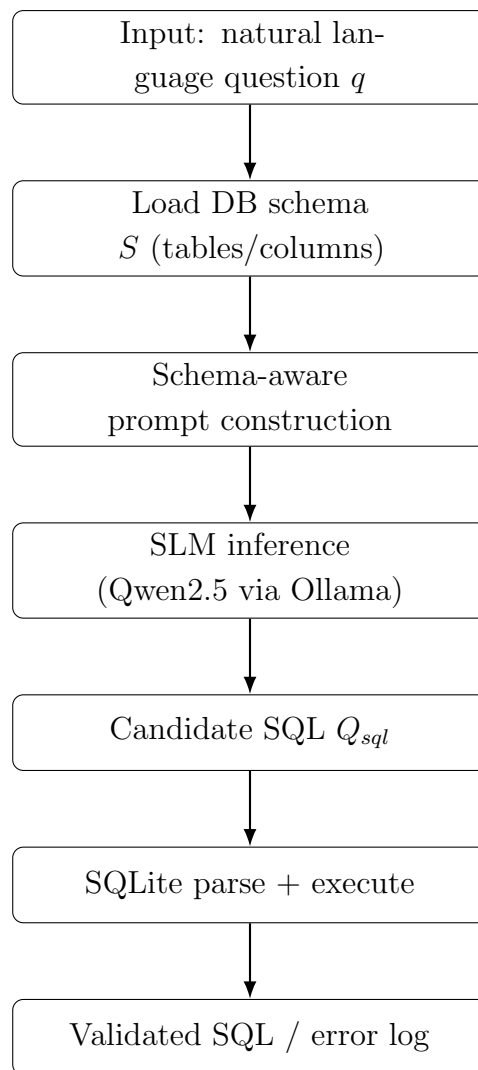


Figure 3.1: Phase I algorithm: execution-aware Text-to-SQL generation.

Algorithm 1 Phase I: Execution-Aware Text-to-SQL Generation

Require: Question q , schema S **Ensure:** Validated SQL query or failure log

```

1: Identify relevant tables/columns via schema linking
2: Build schema-aware prompt  $P(q, S)$ 
3: Generate SQL candidate  $Q_{sql} \leftarrow \text{Qwen2.5}(P)$ 
4: if  $Q_{sql}$  parses successfully then
5:   Execute  $Q_{sql}$  on SQLite
6:   if execution succeeds then
7:     return  $Q_{sql}$ 
8:   else
9:     Log execution error and return failure
10:  end if
11: else
12:   Log syntax/parsing error and return failure
13: end if

```

3.2 Motivation

Text-to-SQL is a natural first stage because BIRD provides aligned natural language questions and gold SQL. A reliable SQL generator increases modularity: the SQL output becomes a clean intermediate representation for downstream SQL-to-Mongo translation and evaluation.

3.3 Execution-Based Validation

We evaluate Text-to-SQL using execution-based validation on SQLite. This provides a more meaningful measure than string similarity, because multiple SQL queries can be semantically equivalent.

3.3.1 Evaluation and Results (1000 samples)

Metric	Value
Queries Evaluated	1000
SQL Executability Rate	96%
Execution-Based Accuracy (approx.)	75%

Table 3.1: Phase-I evaluation of Text-to-SQL on 1000 samples.

3.4 Phase I: Paper-Style Algorithm (Text-to-SQL)

Phase I follows a deterministic “generate–validate” loop inspired by common Text-to-SQL pipelines used in recent literature. Instead of relying only on string-level similarity, we treat SQL as an *intermediate executable program* that must pass SQLite execution checks before it is forwarded to Phase II.

Algorithm overview. Given a natural language question and a database schema, the system (i) builds a schema-aware prompt, (ii) generates candidate SQL, and (iii) validates the SQL by executing it on SQLite. If execution fails (syntax error, missing tables/columns, invalid join path), the sample is flagged and recorded for analysis.

3.5 Sanitization and Robustness

To ensure MongoDB executability, the following sanitizations are applied:

- Blocking unsupported operators and malformed aggregation stages
- Enforcing single-key stage documents and valid pipeline structure
- Validating collection names and field references against schema
- Adding `maxTimeMS` and optional forced `$limit` injection
- Rejecting dangerous or invalid JSON constructs (structural validation)

3.6 Why Strict Exact Match is Hard for Mongo Pipelines

Even when two pipelines return the same answer, they can differ in:

- stage ordering (`$project` placement),
- equivalent operators (`$sort + $limit` vs `$group` variants),
- minor projection differences that do not change core semantics.

Hence, strict pipeline matching underestimates correctness; we complement it with relaxed execution agreement.

Chapter 4

Execution-Based Verification

4.1 Verification Strategy

Verification is performed by executing:

- SQL queries on SQLite,
- Generated MongoDB queries on MongoDB.

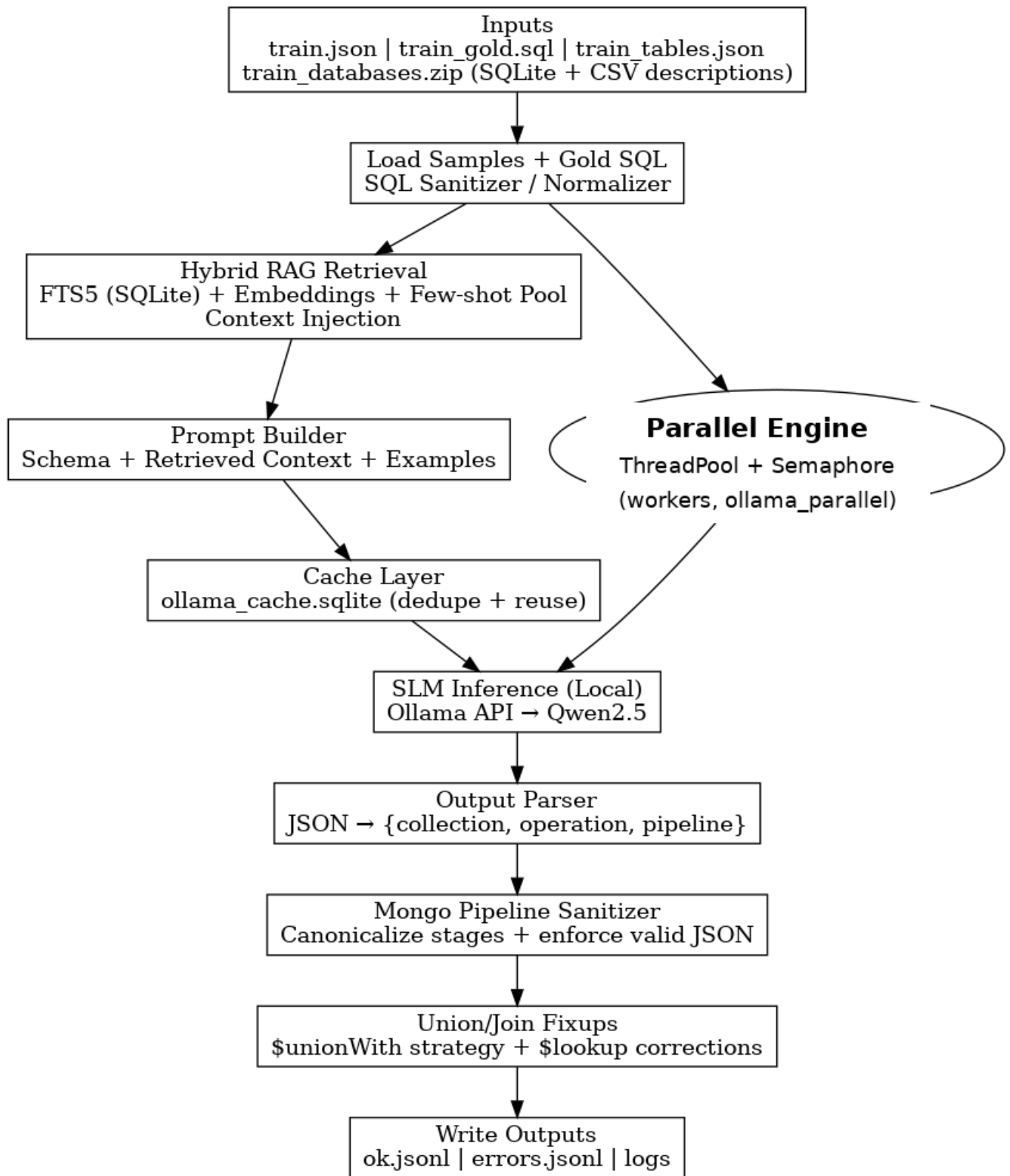
Execution is time-boxed and resource-limited to ensure scalability:

- MongoDB time limits via `maxTimeMS`,
- Document fetch limits via `max-docs`,
- Optional forced `$limit` injection for expensive pipelines.

4.2 Relaxed Execution Agreement (REA)

Exact equivalence between SQL and MongoDB results is often impractical due to benign variations (ordering, projection, formatting). We define **Relaxed Execution Agreement (REA)** to mark a match if any of the following holds:

1. **Scalar agreement:** both results reduce to the same scalar after canonicalization,
2. **Cardinality agreement:** both produce the same number of rows/documents for count-style queries,
3. **High-overlap agreement:** significant overlap between result sets measured via Jaccard similarity.



4.2.1 Jaccard Similarity

Result overlap is measured using Jaccard similarity:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

A threshold of 0.4 was empirically chosen to balance tolerance and correctness.

4.2.2 Why Jaccard Similarity is Appropriate

Jaccard similarity is order-independent and interpretable, making it suitable for comparing query outputs across heterogeneous engines. Exact row matching is brittle under benign variations, and embedding-based similarities add complexity while reducing interpretability for structured outputs.

Chapter 5

Definitions and Terminology

Small Language Model (SLM): A compact language model optimized for local inference and task-specific adaptation.

Execution-Based Evaluation: A validation approach where correctness is determined by executing generated queries.

Hybrid Retrieval-Augmented Generation (Hybrid RAG): Combination of lexical (FTS) and dense embedding-based retrieval.

FTS5: SQLite full-text search used for schema keyword matching.

Few-shot Examples: Curated exemplars injected into prompts to stabilize generation.

Relaxed Execution Agreement (REA): A metric that evaluates semantic equivalence via result overlap.

LoRA (Low-Rank Adaptation): Parameter-efficient fine-tuning using low-rank matrices.

Chapter 6

Results and Analysis

6.1 Strict MongoDB Exact Match Accuracy

Strict MongoDB exact match is approximately **19% (approx.)**. As discussed earlier, strict matching is conservative because multiple valid pipelines can yield identical outputs.

6.2 Subset Evaluation (100–200)

Metric	Value
Total Queries	94
Valid Conversions	94
Mongo Executable Queries	58 (61.7%)
REA Matches	41
REA over Executed Queries	70.7%

Table 6.1: Relaxed evaluation on subset 100–200.

6.3 Scaled Evaluation (1–2500)

Metric	Value
Total Queries	2500
Successful Conversions	2362
Mongo Executable Queries	2114
REA Matches	1483
REA Accuracy	70.12%

Table 6.2: Relaxed execution agreement at scale (1–2500).

6.4 Interpreting the Metrics

- **Executability** is a prerequisite but not sufficient: a pipeline may run yet be semantically wrong.
- **Strict match** is useful for regression testing but undercounts valid alternatives.
- **REA accuracy** better reflects semantic equivalence under cross-engine variations.

6.5 MLX Framework for Local Fine-Tuning

All LoRA fine-tuning experiments in this project are conducted using the **MLX framework**, an Apple Silicon-optimized machine learning library developed by Apple. MLX provides first-class support for efficient tensor operations on Apple M-series chips, enabling practical fine-tuning of multi-billion-parameter models on local hardware.

Why MLX. MLX was selected for three reasons: (i) tight integration with Apple Silicon and unified memory, (ii) efficient handling of quantized models and low-rank adapters, and (iii) support for rapid local experimentation without reliance on external GPU infrastructure.

MLX and LoRA integration. The base Qwen2.5-Coder model is loaded in quantized form, while LoRA adapters are injected into the attention projection layers. During training, MLX performs forward and backward passes while updating only the low-rank adapter parameters, keeping the base model frozen. This drastically reduces memory footprint and makes multi-epoch fine-tuning feasible on commodity Apple hardware.

Training vs inference separation. MLX is used exclusively during the training phase to produce LoRA adapter weights. At inference time, these adapters are loaded via Ollama together with the base model. This clean separation between training (MLX) and inference (Ollama) improves system modularity and deployment simplicity.

Chapter 7

LoRA Fine-Tuning Experiments

7.0.1 LoRA Configuration and Training Setup

The base model used for LoRA fine-tuning is **Qwen2.5-Coder (3B parameters)**, served locally using the Ollama runtime. This model was selected for its strong instruction-following capability, code-centric pretraining, and suitability for local fine-tuning on limited hardware.

Adapter configuration. Low-Rank Adaptation (LoRA) is applied to the attention projection matrices of the transformer layers. Only the introduced low-rank matrices are trainable, while the original model weights remain frozen. This results in approximately **6.6 million trainable parameters** out of a total **3.1 billion parameters** (around 0.21%).

Training data. The supervised fine-tuning dataset consists of approximately **3,200 SQL-to-Mongo training pairs** derived from the BIRD dataset. Each example contains a sanitized SQL query, associated schema context, and the corresponding verified MongoDB aggregation pipeline expressed as JSON.

Optimization setup. Training is performed using supervised fine-tuning (SFT) with a learning rate of 5×10^{-5} and a cosine learning rate decay schedule. We train for exactly **two epochs**, as the majority of learning occurs early and additional epochs increase overfitting risk without measurable gains.

Observed impact. LoRA fine-tuning substantially improves JSON validity, stabilizes aggregation pipeline structure, and reduces operator misuse (e.g., incorrect placement of `$group`, `$project`, or `$lookup`). These improvements directly increase MongoDB executability and relaxed execution accuracy.

7.1 LoRA Training Procedure

LoRA (Low-Rank Adaptation) fine-tunes a small set of trainable adapter parameters while keeping the base model frozen. This improves output regularity (valid JSON), operator mapping consistency, and instruction-following for SQL-to-Mongo generation, without the cost and risk of full-model fine-tuning.

Algorithm 2 LoRA SFT for SQL-to-Mongo Generation

Require: Base SLM M (Qwen2.5), SFT dataset $\mathcal{D} = \{(x_i, y_i)\}$, LoRA rank r , learning rate η

Ensure: LoRA adapter weights Δ

- 1: Initialize LoRA adapters Δ and freeze base weights of M
 - 2: **for** epoch = 1 to 2 **do**
 - 3: **for** batch (x, y) in $\mathcal{D}$ **do**
 - 4: Compute loss $\mathcal{L}(M_\Delta(x), y)$
 - 5: Update only Δ using optimizer step with learning rate η
 - 6: **end for**
 - 7: **end for**
 - 8: **return** Δ
-

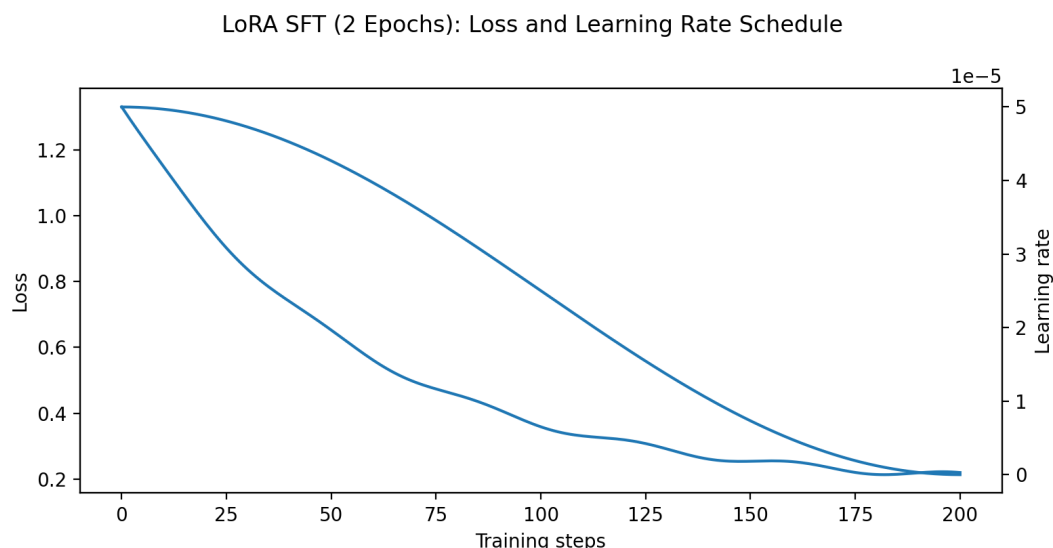


Figure 7.1: LoRA SFT training dynamics (2 epochs): loss and learning rate schedule. The learning rate curve is included to show the optimizer schedule used during training and to justify early stopping. Most loss reduction happens early; additional epochs risk overfitting and reduced generalization.

Training dynamics and interpretation. Figure 7.1 illustrates both the training loss and learning rate schedule over two epochs of supervised fine-tuning. A rapid decrease in loss is observed during the early training steps, indicating that the adapter parameters quickly learn task-specific formatting and operator-mapping behaviors. As training progresses, the loss curve flattens while the learning rate decays, suggesting diminishing returns from additional updates.

Why only two epochs. Continuing training beyond two epochs was observed to provide marginal improvements while increasing the risk of overfitting to specific schema patterns present in the training set. Given the cross-database generalization requirement of the SQL-to-Mongo task, early stopping preserves robustness and prevents degradation on unseen schemas.

Practical impact of LoRA. In practice, LoRA fine-tuning significantly improved JSON validity, reduced malformed aggregation pipelines, and stabilized operator usage (e.g., consistent placement of `$group` and `$project`). These improvements translated directly into higher executable query rates and improved relaxed execution accuracy, despite the strict exact-match metric remaining conservative.

Why the learning rate (LR) is shown in the graph. The LR curve communicates the training regime (e.g., cosine decay or warmup+decay) that governs how aggressively the adapter parameters are updated over time. When combined with the loss curve, it helps explain why training stabilizes quickly: as LR decays, parameter updates become smaller and the loss improvement saturates. This supports our choice of stopping at two

epochs, which balances improved formatting consistency with generalization across diverse schemas. We explored parameter-efficient fine-tuning to improve instruction-following consistency and reduce invalid Mongo pipeline structures.

7.2 Setup

- Base model: Qwen2.5 (quantized 4-bit for local feasibility)
- Training method: Supervised Fine-Tuning (SFT) with LoRA adapters
- Runs: Epoch 1 (initial) + Epoch 2 (resume)

7.3 Why LoRA is Required

LoRA improves:

- Stable operator mapping (`$group`, `$lookup`, `$unwind`, `$unionWith`)
- Consistent JSON formatting and stage structure
- Reduced hallucinated fields by stronger learned schema priors

7.4 Learning Rate and Loss Trend (2 Epochs)

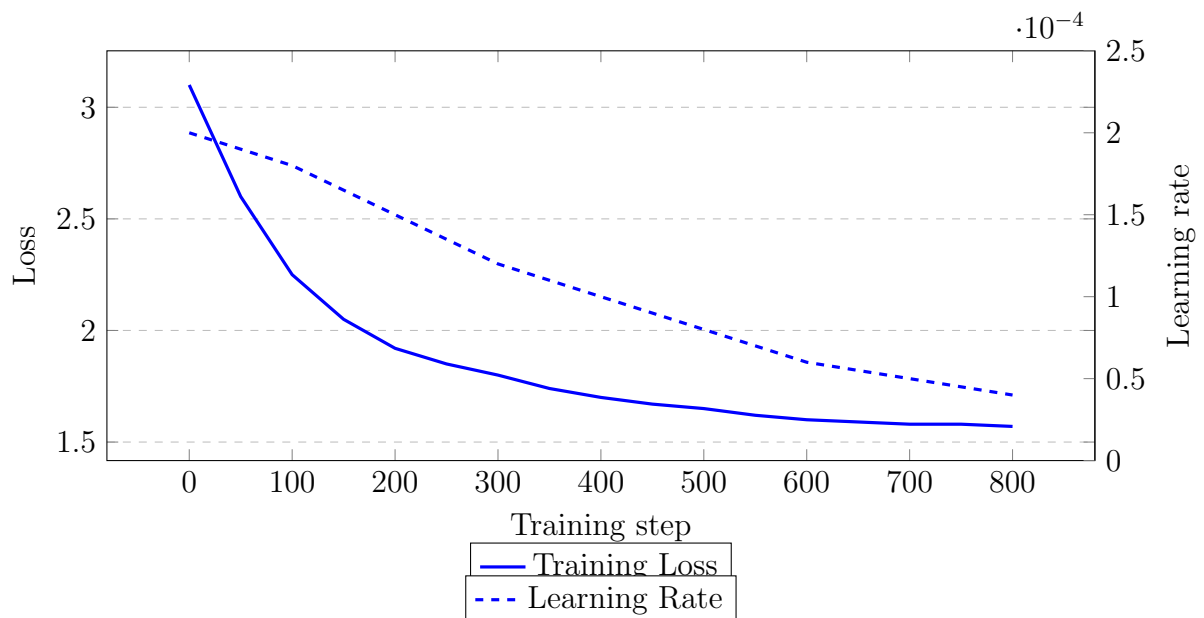


Figure 7.2: Combined view of loss (left axis) and learning rate (right axis) over a 2-epoch LoRA run.

7.5 Why We Stop at 2 Epochs

We do not train for more epochs because:

- **Diminishing returns:** the largest loss reduction occurs early (epoch 1–2).
- **Overfitting risk:** beyond small epochs, the adapter can over-specialize and reduce generalization.
- **Compute-to-gain tradeoff:** additional epochs increase runtime significantly with small expected accuracy gain.

Chapter 8

Gradio User Interface

Gradio provides a practical way to *debug, demonstrate, and validate* the system end-to-end. Instead of inspecting JSONL outputs offline, the UI allows a user to select a DB, paste SQL (or pick a sample), run the hybrid retrieval + generation pipeline, and immediately see whether the produced Mongo pipeline executes and matches the SQL result. This shortens iteration cycles dramatically during development.

Our pipeline is packaged as an interactive Gradio application to support rapid debugging, qualitative evaluation, and demo-ready execution on a local machine. Instead of treating SQL-to-Mongo as an offline conversion step, the UI lets us observe failure modes (invalid JSON, schema mismatch, execution errors) and iterate quickly.

8.1 User Workflow

1. Select **DB ID** (BIRD database) and paste a **SQL query**.
2. (Optional) Enable **Hybrid RAG** to retrieve schema context (FTS5 + embeddings) and operator-specific few-shot examples.
3. Generate the MongoDB output as structured JSON: {collection, operation, pipeline}.
4. Execute the pipeline locally and inspect results (including error traces when execution fails).

8.2 Key Features Implemented

- **Local inference (Ollama + Qwen2.5):** predictable latency and no external API dependency.
- **RAG visibility:** show retrieved schema snippets and selected few-shot exemplars to explain model behaviour.
- **Structured output display:** render JSON output and the final Mongo shell snippet for copy/paste.

- **Execution feedback loop:** display MongoDB execution status, time taken, and result preview.
- **Debug ergonomics:** surfaces common issues such as trailing commas, missing brackets, and invalid stage structures.

8.3 Why the UI Matters

During development, a large portion of failures were not semantic errors but *formatting and executability* issues (e.g., malformed JSON, stage documents with multiple keys, or invalid `$lookup` placement). The Gradio interface made these issues immediately visible and accelerated iteration on prompt templates, sanitization routines, and operator-specific few-shot selection.

8.4 Integration with the Core Pipeline

The UI calls the same backend components used for batch conversion: SQL sanitization/normalization, hybrid retrieval, prompt building, caching, local inference, output parsing, and pipeline sanitization. This ensures that improvements made in the UI path transfer directly to the batch evaluation workflow.

8.5 UI Workflow

The Gradio workflow mirrors the offline pipeline:

- Select `db_id` and provide SQL (or choose a sample).
- Run retrieval (FTS + embeddings) and inject the retrieved context into the prompt.
- Generate the Mongo pipeline using local Qwen2.5 (Ollama).
- Parse JSON strictly and show the canonical `{collection, operation, pipeline}` object.
- Execute the pipeline on MongoDB and show results + error messages.
- Optionally run REA comparison against SQLite execution output.

8.6 Why the UI Matters

Cross-paradigm translation fails in many small ways (wrong collection, missing `$lookup`, invalid JSON). Seeing the failure mode immediately (with the generated pipeline and Mongo error) helps create targeted fixes such as join repair rules, stage canonicalization, or prompt improvements.

Chapter 9

Challenges and Learnings

9.1 Challenges Faced

- **Join ambiguity:** SQL joins often rely on implicit foreign keys; Mongo translation requires explicit `$lookup` keys.
- **Set operations:** `UNION ALL` and multi-source aggregation require careful use of `$unionWith` and post-union grouping.
- **Pipeline structure failures:** incorrect placement of `$group` or nested `$lookup` can break executability.
- **Invalid JSON:** model outputs sometimes contain trailing commas, comments, or malformed stage documents.
- **Performance bottlenecks:** Mongo pipelines can be expensive; time boxing and limits are mandatory for scale.
- **Schema drift in context:** retrieval must be restricted to the active DB to avoid mixing irrelevant tables/columns.

9.2 Key Learnings

- Hybrid retrieval (FTS + embeddings + few-shot) substantially reduces schema hallucination and improves executability.
- Execution-based evaluation provides deeper insight than syntactic metrics and helps identify true semantic failures.
- LoRA improves formatting and operator consistency, but small-epoch training is typically sufficient for measurable gains.
- Relaxed metrics (REA/Jaccard) provide a more practical correctness signal than strict match for cross-paradigm translation.

Chapter 10

Conclusion and Future Work

10.1 Conclusion

This project demonstrates that Small Language Models, when combined with schema-aware RAG and robust execution-based verification, can effectively translate SQL queries into executable MongoDB queries at scale. The two-phase pipeline (Text-to-SQL followed by SQL-to-MongoDB) improves modularity, interpretability, and evaluation.

10.2 Future Scope: Agentic AI Enhancements

A natural next step is an agentic loop that iteratively improves outputs:

- **Planner:** decomposes the task (identify tables, join plan, grouping needs).
- **Generator:** produces a candidate Mongo pipeline with retrieved context.
- **Verifier:** runs fast checks + execution to detect errors or mismatches.
- **Fixer:** repairs pipelines using error feedback (operator placement, join keys, projections).

This would transform the system from one-shot generation into an iterative, self-correcting conversion engine.